CodInClub
Powered By BridgeLabz

Week 1 Assignment — 5 Problems

---

1. The Exam Hall Seat Duplication Checker

Scenario

The Examination Cell manages seat allocation across a large exam hall for hundreds of students. Before an exam begins, invigilators must confirm that no seat number has been assigned to two different students by mistake — a data-entry slip in the seating spreadsheet could otherwise seat two students in the same chair. The system needs to scan the full list of assigned seat numbers and flag any duplicates before the exam starts.

Task

- Accept an array of seat numbers (integers) assigned to students in a hall.
- Compare every seat number against every other seat number to check for duplicates (do not use any Collections class — arrays and loops only).
- If one or more duplicates are found, print the duplicated seat number(s).
- If no duplicates exist, print a clear confirmation message.

Suggested method signature(s): void checkDuplicateSeats(int[] seatNumbers)

Sample Input / Output

Input	Output
{101, 102, 103, 102, 105}	Duplicate Seat Number Found: 102
{101, 102, 103, 104, 105}	No Duplicate Seats Found

Concepts covered: Arrays, nested loops, conditional logic, basic output formatting.

2. The Typing Speed Test Accuracy Checker

Scenario

An online typing-practice website shows users a fixed passage and asks them to retype it as quickly and accurately as they can. Once the user submits their attempt, the system must compare it character by character against the original passage and report exactly how accurate the attempt was — along with the position of the very first mistake — so the user can see precisely where their typing went wrong.

Task

- Accept two strings of equal length: the original passage and the user's typed text.
- Compare the two strings character by character using their positions.
- Count how many characters match at the same position.
- Calculate the accuracy percentage: (matched characters ÷ total characters) × 100.
- Print the accuracy percentage and the position of the first mismatch (or a message confirming there were no mismatches).

Suggested method signature(s): void checkTypingAccuracy(String original, String typed)

Sample Input / Output

Input	Output
original="hello world", typed="hello worlt"	Matched: 10/11 | Accuracy: 90.91% | First Mismatch at position 11 ('d' vs 't')
original="coding", typed="coding"	Matched: 6/6 | Accuracy: 100.00% | No Mismatches

Concepts covered: String traversal, charAt(), loops, conditional logic, percentage calculation.

3. The Traffic Signal Streak Analyzer

Scenario

The city traffic control department logs the color shown by a signal every minute using single letters — 'R' for red, 'Y' for yellow, 'G' for green. Engineers suspect a malfunctioning signal at one junction might be getting "stuck" on one color for unusually long stretches. They need a tool that scans a day's log and reports the longest continuous streak of the same color, so they know exactly which signal to inspect first.

Task

- Accept a string representing a sequence of signal readings (e.g., "RRGGGYRR").
- Scan through the string and track the length of each streak of consecutive identical characters.
- Keep a running record of the longest streak found so far — both its color and its length.
- Print the color and length of the longest streak.

Suggested method signature(s): void findLongestStreak(String signalLog)

Sample Input / Output

Input	Output
"RRGGGYRR"	Longest Streak: 'G' repeated 3 times
"RRRRYYGG"	Longest Streak: 'R' repeated 4 times

Concepts covered: String traversal, character comparison, loops, tracking a running maximum.

4. The Warehouse Inventory Balancer

Scenario

A retail warehouse stores the same product categories across two storage sections, Section A and Section B. Before the monthly stock report is generated, the inventory team wants to confirm both sections hold matching total quantities (to catch data-entry mismatches) and also identify the single highest-quantity item across the whole warehouse.

Task

- Accept two integer arrays of equal length representing item quantities in Section A and Section B.
- Compute the total quantity held in each section.
- Compare the two totals and print whether the sections are "Balanced" or "Not Balanced".
- Scan both arrays to find the single highest quantity value and report which section and index it was found at.

Suggested method signature(s): void analyzeInventory(int[] sectionA, int[] sectionB)

Sample Input / Output

Input	Output
sectionA={20,15,30}, sectionB={25,10,30}	Section A Total: 65 | Section B Total: 65 | Status: Balanced | Highest Quantity: 30 (Section A, Item 3)

Concepts covered: Arrays, loops, sum accumulation, conditional comparison, tracking maximum with its index.

5. The Movie Review Word Length Profiler

Scenario

A movie-review platform's moderation tool scans newly submitted reviews and profiles the length of the words used in them — reviews stuffed with unusually many very short or very long words are more likely to be spam or bot-generated, so moderators want a quick word-length breakdown before a review is allowed to go live.

Task

- Accept a movie review as a single string input.
- Split the review into individual words.
- Classify each word as Short (1–4 letters), Medium (5–8 letters), or Long (9+ letters).
- Count how many words fall into each category.
- Print the final counts for Short, Medium, and Long words.

Suggested method signature(s): void classifyWordLengths(String review)

Sample Input / Output

Input	Output
"This movie was absolutely fantastic and thrilling"	Short: 3 | Medium: 1 | Long: 3

Concepts covered: String splitting (split()), loops, conditional logic, counting/categorization.